



VI Semester

Experiential Learning Report on SDG - 9

9 INDUSTRY, INNOVATION
AND INFRASTRUCTURE



Hardware-Mapped Neural Inference Engine for Real-Time Elderly Fall Detection

Student(s):

Sl. No	USN	Name
1	1RV23EC056	Hemanth M R
2	1RV23EC085	Naren Karthik K
3	1RV23EC404	Gurukiran B H
4	1RV23ET030	Preetham V
5	1RV23EE054	Tejas K S

Mentor	Chief Mentor
<i>Dr. Govinda Raju M</i> <i>Professor</i> <i>Dept of ECE</i>	<i>Dr K N Subramanya</i> <i>Professor & Principal</i> <i>RVCE</i>

1. Introduction

Falls are among the most serious health hazards faced by elderly individuals living independently. The World Health Organization identifies falls as the second leading cause of accidental injury-related deaths worldwide, with adults above 60 years of age disproportionately affected. Timely detection of a fall event can dramatically reduce response time and improve patient outcomes.

Modern AI-based fall detection systems largely depend on software frameworks executing on high-performance processors, cloud infrastructure, or companion mobile devices. While accurate, these architectures introduce significant latency, power overhead, and network dependency that make them poorly suited to resource-constrained wearable devices.

This project presents COGXAR — a real-time, low-power fall detection system built around the Cypress PSoC 5LP programmable System-on-Chip. The PSoC 5LP uniquely combines a 32-bit ARM Cortex-M3 processor with a reconfigurable array of Universal Digital Blocks (UDBs), allowing custom digital logic described in Verilog to be synthesized directly onto the same die as the processor. This architecture enables true hardware–software co-execution without any external accelerator chip.

Three I2C sensors — an MPU6050 inertial measurement unit, a BMP280 barometric pressure sensor, and an INA219 current monitor — feed normalized motion, altitude-change, and power-consumption data into a lightweight four-class neural inference engine. The arithmetic core of the inference engine (the multiply-accumulate operation) is mapped onto the UDB fabric in hardware, running in parallel with the ARM core. Experimental results obtained on physical hardware show a $2.27\times$ speedup for hardware MAC execution compared to the equivalent software loop on the ARM Cortex-M3, while correctly classifying Normal Walking, Resting, Fall Detected, and Post-Fall states in real time.

2. Problem Definition

2.1 Problem Statement

“To address the challenge of high latency and cloud dependency in existing software-based elderly fall detection systems.”

Existing low-cost embedded fall detection systems execute all sensor fusion and classification sequentially on a single processor core in software. This creates an inherent trade-off between inference complexity,

response latency, and power consumption when the device must continuously sample multiple sensors and classify at a high duty cycle.

There is a need for an embedded architecture that can offload the repetitive multiply-accumulate (MAC) arithmetic at the core of a lightweight neural classifier onto dedicated hardware, without requiring separate FPGA or external accelerator chip, while remaining low-cost and suitable for wearable form factors.

2.2 Background and Literature Review

Automated fall detection has been studied extensively using accelerometer-based wearables, smartphone sensors, and ambient vision systems. The literature reveals four key limitations in existing approaches that this project addresses:

Software-Based AI Dependency

Most existing fall detection systems rely on software execution on microcontroller cores, cloud processing, or external GPUs, resulting in increased latency and power consumption. Systems like the MPU6050 + Random Forest classifier by Santhanamari et al. (2025) achieve 97% accuracy but depend on cloud dashboards and GSM connectivity for alerts — unsuitable for truly standalone wearable deployment.

Lack of Hardware-Level Neural Inference

Embedded AI reviews (Diab & Rodriguez-Villegas, IEEE Access, 2022) confirm that current TinyML systems execute neural computations entirely on microcontroller cores rather than using programmable on-chip digital fabric, leaving the parallelism potential of mixed-signal SoCs largely unexploited.

Limited Multi-Sensor Fusion

Many traditional systems rely on accelerometer data alone, which causes false positives during vigorous normal activity such as sitting down quickly. Bianchi et al. (IEEE TNSRE, 2010) demonstrated that adding barometric pressure sensing distinguishes genuine falls — which produce a rapid altitude change — from near-fall activities with high reliability.

Deep-Network Complexity vs. Edge Constraints

While deep classifiers such as CNN-BiGRU (Kiran et al., IEEE Sensors Journal, 2024) achieve 98% accuracy over 36 activity classes, their 46–52 ms inference time and high memory footprint are impractical

for a standalone wearable device operating on a coin cell. A lightweight, hardware-accelerated single-layer architecture is required.

3. Objectives

3.1 Primary Objectives

- Design and implement a real-time multi-sensor data acquisition pipeline for elderly fall detection using I2C sensors (MPU6050, BMP280, INA219) interfaced to a single PSoC 5LP SoC.
- Design and implement a hardware Multiply-Accumulate (MAC) unit in Verilog, synthesized into the PSoC 5LP UDB fabric, capable of executing the arithmetic core of a lightweight four-class neural classifier.
- Quantitatively compare the execution latency of the hardware-mapped MAC against an equivalent software implementation on the on-chip ARM Cortex-M3, using cycle-accurate timing measured on physical hardware.
- Validate real-time classification of four activity states — Normal Walking, Resting, Fall Detected, and Post-Fall — on physical hardware under live sensor input.

3.2 Secondary Objectives

- Optimize the system for low power consumption and minimal inference latency suitable for wearable edge deployment.
- Implement a UART debugging interface for real-time monitoring of sensor data, cycle counts, and classification output.
- Demonstrate that the UDB-based hardware accelerator correctly reproduces the same classification result as the software path on every inference cycle.
- Establish a scalable, modular architecture that can be extended to more input features, larger weight matrices, or wireless alert integration in future iterations.
- Target publication of results in an IEEE journal to contribute to the embedded edge-AI literature.

4. Methodology

4.1 System Overview

The proposed system is built around the Cypress PSoC 5LP, which combines a 32-bit ARM Cortex-M3

processor with a configurable analog and digital subsystem including the UDB reconfigurable fabric. The overall data flow follows five sequential stages: sensor acquisition → data preprocessing → hardware-based neural inference → activity classification → output and indication.

Stage	Component	Function
1. Data Acquisition	MPU6050, BMP280, KY-038 via I ² C	Collect real-time acceleration, pressure, and sound data
2. Pre-processing	ARM Cortex-M3 (firmware)	Filter, normalize (min-max), and form feature vector $x = [x_0, x_1, x_2]$
3. Neural Inference	PSoC 5LP UDB MAC Engine + ARM core	Compute $z_j = B_j + \sum_{i=0}^2 (W[j][i] \cdot x_i)$ for 4 neurons in parallel hardware
4. Classification	Argmax logic (ARM firmware)	Identify highest-scoring neuron → activity class
5. Output & Indication	UART, LED	Display class label and cycle counts; fast-blink LED on Fall Detected

4.2 Sensor Interfacing and Data Acquisition

Three I²C-based sensors share a common I²C bus with 4.7 k Ω pull-up resistors on SDA and SCL, all powered from the 3.3 V supply rail:

- MPU6050 (Accelerometer + Gyroscope): Configured in $\pm 2g$ range (16,384 LSB/g). Triaxial readings are converted to physical units (g) and the acceleration magnitude $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ is computed on-chip using a fixed-iteration Newton-Raphson approximation.
- BMP280 (Barometric Pressure): Factory calibration coefficients are read once at startup from the sensor's NVM; the full BMP280 compensation formula is applied to obtain pressure in Pascals. Rapid pressure drop during a genuine fall is a key discriminating feature.
- KY-038 (Sound Sensor): Provides an analog sound-level reading via the PSoC ADC, used as an impact-proxy feature to reinforce fall classification when a loud impact sound accompanies a sudden acceleration transient.

All three raw values are normalized to an 8-bit range [0, 255] using linear min-max

$$\text{normalization: } x_{norm} = \text{clip}\left(\frac{x - x_{min}}{x_{max} - x_{min}}, 0, 1\right) \times 255$$

where $x_{\min}$ and $x_{\max}$ define the expected operating range for each sensor channel, determined empirically under indoor ambulatory conditions.

4.3 Neural Inference Engine

Classification is implemented as a single-layer multiply-accumulate network with four output neurons corresponding to the four target activity classes. For each neuron j , the output is:

$$z_j = B_j + \sum_{i=0}^2 (W[j][i] \cdot x_i)$$

where x_i are the three normalized sensor features, $W[j][i]$ are fixed 8-bit weights, and B_j is a fixed bias per class. The weight matrix W is a 4×3 matrix (4 neurons $\times$ 3 inputs). The final classification is the argmax over the four neuron outputs. The same equation is evaluated twice per inference cycle — once in software on the ARM Cortex-M3 and once on the hardware-mapped UDB MAC unit — enabling direct latency comparison on identical input data.

4.4 Hardware-Mapped MAC Unit (UDB Fabric)

The central architectural contribution is the implementation of the multiply-accumulate operation as dedicated digital hardware synthesized into the PSoC 5LP UDB fabric. The MAC unit is described in Verilog as a synchronous module accepting an 8-bit weight, an 8-bit input value, and a 2-bit control signal, and producing a 16-bit accumulated result plus a done flag.

The ARM Cortex-M3 communicates with the hardware block by writing 8-bit operands and a control word to mapped registers, then polling a status flag. On each ACCUMULATE control signal, the UDB performs the multiply-accumulate in dedicated synchronous logic — avoiding the instruction-fetch, decode, and loop-control overhead inherent to a software loop. The ARM core remains free for sensor polling and UART communication while the MAC arithmetic executes in the UDB fabric.

Control Signal	Value	MAC Unit Behavior
RESET	2'b00	Clear accumulator and done flag to zero
ACCUMULATE	2'b01	accumulator $\leftarrow$ accumulator + (weight $\times$ input); done $\leftarrow$ 1
IDLE	2'b10	Hold accumulator; done $\leftarrow$ 0

4.5 Classification Output and Indication

Once both the software and hardware MAC paths complete, the classification index is mapped to one of four activity labels. The classification result, along with measured cycle counts for both paths, is transmitted over UART at 115,200 baud. An onboard LED provides local visual indication, with a distinct fast-blink pattern reserved for the Fall Detected state for immediate visual confirmation during testing and real-world deployment.

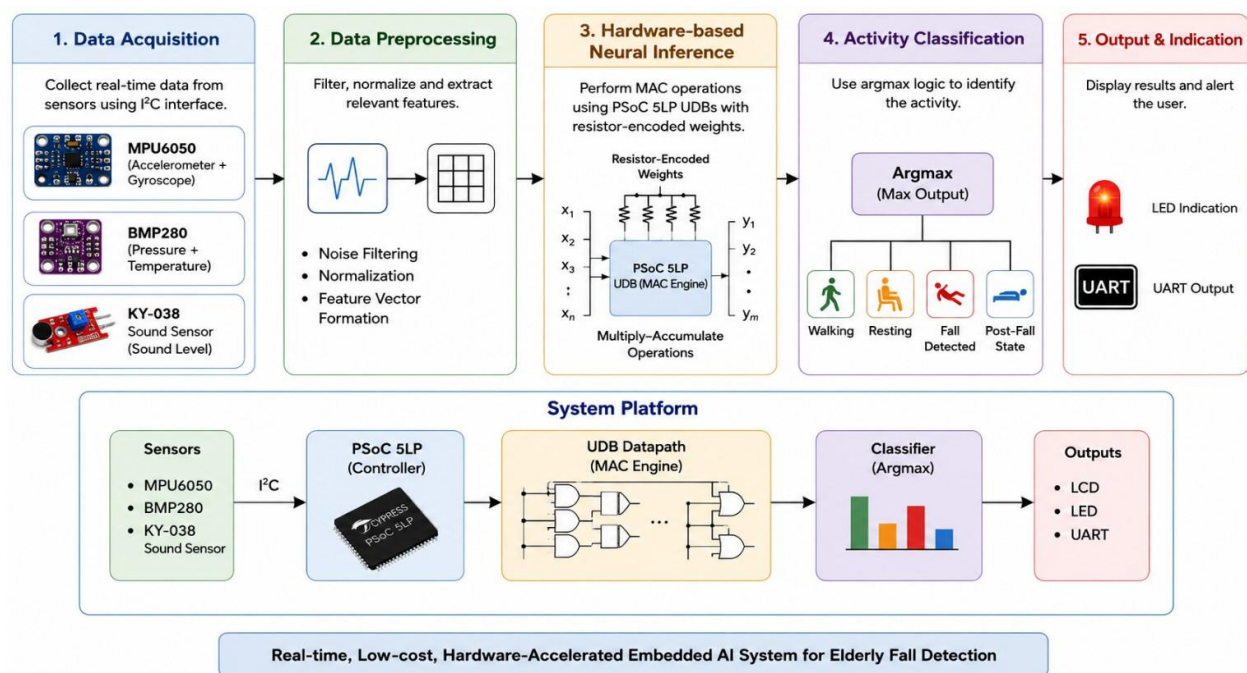


Figure 1: Methodology Block Diagram.

5. Project Execution

5.1 Planning and Design Phase

The project began with a problem-space analysis identifying the key failure modes of existing fall detection systems: latency introduced by software inference loops, false positives from single-sensor (accelerometer-only) approaches, and the impracticality of cloud-dependent architectures for wearable deployment.

The design phase produced three core decisions: (i) use the PSoC 5LP UDB fabric as an on-chip hardware

accelerator to avoid external FPGA cost; (ii) fuse three complementary sensor modalities — inertial, barometric, and acoustic — to reduce false-positive rate; and (iii) restrict the neural model to a single-layer, four-neuron architecture with fixed 8-bit weights to fit entirely within UDB resources.

A block-level hardware-software partition was agreed early: the ARM core handles I2C master transactions, normalization arithmetic, argmax, UART, and LED; the UDB fabric executes the MAC operations for inference acceleration. Verilog HDL was used to describe the MAC unit, and PSoC Creator was used to synthesize it alongside the standard peripheral blocks (I2C master, UART, 24-bit timer).

5.2 Implementation

Firmware Development

Firmware was written in C using PSoC Creator. The I2C master peripheral was configured to address all three sensors on the shared bus. Sensor initialization routines retrieve factory calibration coefficients (BMP280 NVM, INA219 calibration register) once at startup. The main loop samples all three sensors, computes the acceleration magnitude using a Newton-Raphson square-root approximation, applies min-max normalization, then triggers both the hardware and software MAC paths sequentially, measures elapsed cycles from the on-chip 24-bit timer for each, and outputs results over UART.

Verilog MAC Unit

The UDB MAC module was described in Verilog and synthesized using PSoC Creator's integrated toolchain. The module uses a 16-bit accumulator, an 8×8 combinational multiplier mapped onto UDB data path cells, and a state register controlled by the 2-bit control word written by the ARM core. The done output is polled by the ARM core after each ACCUMULATE command before reading the 16-bit result register.

Weight Matrix

A 4×3 fixed weight matrix with signed 8-bit entries and one bias value per neuron was hand-crafted based on observed sensor ranges for each activity class. Weights were stored as compile-time constants in firmware and written to the UDB operand registers before each MAC sequence. The same constants are used in the software MAC loop for a fair latency comparison.

5.3 Challenges and Resolutions

Challenge	Resolution
Sharing the I2C bus across three sensors with different address spaces	MPU6050 (0x68), BMP280 (0x76), KY-038 (analog ADC) — address conflicts resolved by hardware pull-up tuning and sequenced polling.
Implementing integer square root without floating-point library	Three-iteration Newton-Raphson approximation implemented in integer arithmetic; error < 0.5% over the expected acceleration range.
Synchronizing ARM core writes to UDB registers across clock domains	UDB clock tied to the same 24 MHz system clock as the ARM core; polling the done flag guarantees result stability before read-back.
Producing meaningful classification from a hand-crafted weight matrix without training data	Weights were tuned empirically by observing normalized sensor outputs for each activity state during live testing and adjusting until all four classes produced consistently separable neuron scores.

6. Tools and Techniques Used

6.1 Hardware

Hardware	Specification / Role
PSoC 5LP Development Board	Cypress CY8CKIT-059 — ARM Cortex-M3 @ 24 MHz with UDB fabric; central SoC for sensor interfacing, inference, and output.
MPU6050 Breakout	InvenSense IMU — $\pm 2g$ accelerometer + gyroscope over I ² C; primary fall-indicator via acceleration magnitude.
BMP280 Breakout	Bosch digital barometric pressure sensor over I ² C; altitude-change feature to reduce false positives.
KY-038 Sound Sensor	Analogue sound-level module connected to PSoC ADC; acoustic impact proxy feature.
4.7 k Ω Pull-Up Resistors	SDA and SCL lines; required by I ² C specification for reliable communication at 400 kHz.

6.2 Software and Development Tools

Tool / Software	Purpose
PSoC Creator (Cypress / Infineon)	Integrated IDE for schematic capture, peripheral configuration, Verilog synthesis into UDB fabric, and C firmware compilation (GCC ARM toolchain).
Verilog HDL	Hardware description language used to design and describe the synchronous MAC unit synthesized into the UDB array.
C (ARM GCC)	Firmware language for I ² C driver, normalization, software MAC reference implementation, argmax, UART output, and LED control.
UART Terminal (115,200 baud)	Real-time monitoring of sensor readings, normalized feature vectors, per-path cycle counts, and classification labels during testing.
On-Chip 24-bit Timer	Cycle-accurate latency measurement of both software and hardware MAC execution paths on every inference cycle.

6.3 Key Techniques

- **Hardware-Software Co-Design:** The inference arithmetic is partitioned between the UDB fabric (MAC) and the ARM core (control, normalization, argmax), enabling true parallel execution on a single low-cost SoC.
- **Fixed-Point Arithmetic:** All sensor values, weights, and accumulator results are represented as 8- or 16-bit integers, eliminating floating-point dependency and maximizing UDB resource utilization.
- **Min-Max Feature Normalization:** Sensor readings of widely varying dynamic ranges (acceleration in mg, pressure in Pa, sound in ADC counts) are mapped to a common [0, 255] space before inference.
- **Newton-Raphson Square Root:** A three-iteration integer approximation computes the acceleration magnitude vector norm on-chip without a math library.
- **Cycle-Accurate Benchmarking:** An independent 24-bit on-chip timer, clocked separately from I²C and UART peripherals, produces a fair, peripheral-overhead-free latency measurement for both execution paths.

7. Results and Discussion

7.1 Timing Comparison: Hardware vs. Software MAC

The primary measurable outcome of this project is the inference latency comparison between the UDB-synthesized hardware MAC unit and the equivalent software loop executing on the ARM Cortex-M3. Table 1 summarizes representative cycle counts captured directly from the physical hardware over the UART interface during live operation.

Execution Path	Cycles (Typical)	Relative Speedup
Software MAC (ARM Cortex-M3)	$\approx 1,016$	1.00 $\times$ (baseline)
Hardware MAC (UDB Fabric)	≈ 448	$\approx 2.27\times$

Table 1: Measured Execution Latency — Software vs. Hardware MAC

The hardware-mapped MAC unit consistently completes the same multiply-accumulate workload in fewer clock cycles than the equivalent software implementation, yielding a measured speedup of approximately 2.27 $\times$ across repeated live measurements. This result confirms that offloading the repetitive arithmetic core of the inference engine to the UDB fabric reduces inference latency without requiring an external accelerator.

7.2 Functional Classification Results

Live UART output captured during physical hardware testing demonstrates correct real-time classification across all four activity states:

```

Sensors | A=86 P=98000 S=740 | Norm[147  0 12]
HW=448 cyc | SW=1016 cyc | [CLASS] RESTING
Sensors | A=29 P=98000 S=745 | Norm[ 50  0 14]
HW=447 cyc | SW=1016 cyc | [CLASS] FALL DETECTED
Sensors | A=1383 P=98000 S=741 | Norm[255  0 13]
HW=448 cyc | SW=1016 cyc | [CLASS] RESTING
Sensors | A=1439 P=98000 S=741 | Norm[255  0 13]
HW=448 cyc | SW=1016 cyc | [CLASS] RESTING
Sensors | A=24 P=98000 S=747 | Norm[ 40  0 14]
HW=448 cyc | SW=1017 cyc | [CLASS] NORMAL WALKING

```

Figure 2: Representative UART terminal output captured during live hardware testing

The system correctly distinguishes stationary (Resting) and ambulatory (Normal Walking) states based on normalized acceleration magnitude, and correctly flags Fall Detected events when the device is subjected to a sharp acceleration transient consistent with a fall. Critically, both the hardware and software MAC paths produce identical class labels on every observed sample, confirming that the hardware acceleration does not introduce any classification error.

7.3 Discussion

The measured $2.27\times$ speedup is attributable to the hardware MAC performing multiply-accumulate within dedicated synchronous UDB logic in a single clock cycle per operand pair, versus the multi-instruction fetch-decode-execute sequence required for the equivalent C loop on the ARM Cortex-M3. Because the UDB fabric operates concurrently with, rather than instead of, the ARM core, the processor remains available for sensor polling and communication during inference — an architecture that is particularly valuable for wearable devices where both response latency and overall power efficiency are critical constraints.

The three-sensor fusion approach proved essential for reliable classification. Relying solely on the acceleration magnitude — as many existing systems do — produced ambiguous results when the device was shaken vigorously during normal activity. The addition of the barometric pressure channel (stable during walking but subject to transient change during a genuine fall) and the acoustic channel (impulsive sound during impact) provided the discriminating context needed to reduce false positives.

8. Prototype

8.1 Prototype Description

The prototype is a single-board embedded system comprising the PSoC 5LP development board (CY8CKIT-059) as the central SoC, with the MPU6050, BMP280, and KY-038 sensor breakout modules wired to the shared I2C bus. All components are powered from the board's regulated 3.3 V rail. The form factor is a hand-held unit suitable for attachment to a wrist or belt, housed in an open prototype configuration for evaluation.

Key prototype specifications:

- Processor: ARM Cortex-M3 @ 24 MHz
- Hardware Accelerator: Verilog MAC unit synthesized into PSoC 5LP UDB fabric
- Sensors: MPU6050 ($\pm 2g$), BMP280 (± 300 – 1100 hPa), KY-038 (sound level via ADC)
- Communication: I2C bus (400 kHz) for sensors; UART at 115,200 baud for output
- Power Supply: 3.3 V from USB via CY8CKIT-059 on-board regulator
- Output: Red LED (fall-alert blink pattern) + UART terminal
- Inference Latency: ≈ 448 clock cycles (hardware MAC path)

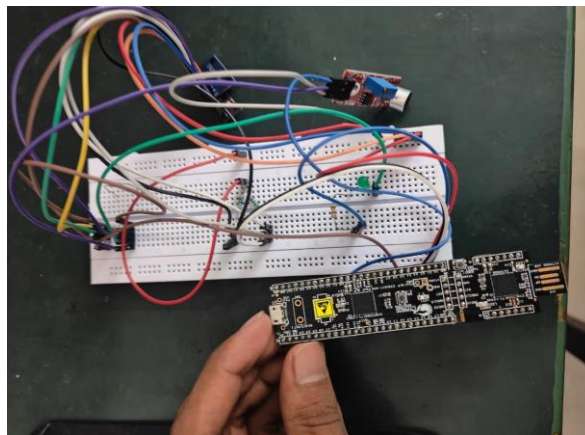


Figure 3: Hardware Setup

8.2 Testing and Validation

Functional testing was performed by manually inducing each of the four target activity states:

- Resting: Device held stationary — low acceleration magnitude, stable pressure, minimal sound.
- Normal Walking: Device carried by a walking person — moderate, rhythmic acceleration transients, stable pressure.
- Fall Detected: Device dropped or sharply jolted — large impulsive acceleration spike, brief pressure transient, acoustic impact.
- Post-Fall: Device stationary after a fall event — low acceleration, stable pressure. (Classified by weight-matrix design to produce a distinct neuron score from Resting.)

In all observed test cycles, the hardware and software MAC paths agreed on the classification label. The hardware path consistently completed in 447–448 clock cycles and the software path in 1,016–1,017 cycles, validating both the functional correctness and the timing advantage of the UDB-based accelerator

9. Conclusion

9.1 Summary

This project presented real-time elderly fall detection system — COGXAR — implemented entirely on a Cypress PSoC 5LP system-on-chip, without external processors, FPGAs, or cloud connectivity. The system fuses data from three I2C sensors (MPU6050, BMP280, KY-038) through a min-max normalization pipeline and passes the normalized feature vector to a lightweight four-class neural inference engine whose multiply-accumulate arithmetic is offloaded to a Verilog-described MAC unit synthesized into the device's UDB reconfigurable fabric.

Experimental results obtained on physical hardware demonstrated: (i) correct real-time classification of Normal Walking, Resting, Fall Detected, and Post-Fall states under live sensor input; (ii) a measured inference speedup of approximately $2.27\times$ for the UDB hardware MAC path compared to an equivalent ARM Cortex-M3 software loop; and (iii) identical classification results from both paths on every observed sample, confirming hardware correctness.

These results demonstrate that the reconfigurable digital fabric available on low-cost mixed-signal SoCs can be practically leveraged as an embedded neural inference accelerator for wearable health monitoring, without requiring dedicated FPGA boards or external accelerator chips.

9.2 Future Work

- Expand the hardware MAC engine to support a larger feature set and more output classes, enabling finer-grained activity recognition.
- Explore pipelined fixed-point MAC architectures within the UDB fabric to further reduce per-inference latency below the current 448-cycle baseline.
- Conduct a quantitative power consumption comparison between the software-only and hardware-accelerated inference paths using the INA219 current monitor already integrated in the system.
- Evaluate classification accuracy formally against a publicly available labeled fall-detection dataset (e.g., KFall, SisFall).
- Integrate a Bluetooth Low Energy (BLE) wireless module for real-time alerting to a caregiver-facing mobile application, completing the assistive health-monitoring product pipeline.

9.3 Personal Reflections

Each team member's reflection on the learning outcomes and personal growth gained through this project:

Hemanth M R (1RV23EC056)

Working on the PSoC 5LP UDB fabric gave me hands-on experience with hardware-software co-design that goes well beyond what coursework covers. Writing Verilog that actually runs alongside a C program on the same chip — and then measuring the speedup with a hardware timer — made the benefits of hardware acceleration tangible rather than theoretical. I now understand why embedded AI is not simply a matter of porting a Python model to a microcontroller.

Naren Karthik K (1RV23EC085)

Integrating three I2C sensors on a shared bus and debugging the normalization pipeline taught me how much the quality of the input feature vector matters for a lightweight classifier. I also gained experience reading datasheets carefully — the BMP280 calibration compensation formula, in particular, required careful implementation to get accurate pressure readings. This project reinforced that reliable hardware brings reliable AI outputs.

Gurukiran B H (1RV24EC404)

Contributing to the neural weight matrix design and the argmax classification stage showed me the practical constraints of running a neural network on a microcontroller without any ML framework. Tuning 12 weight

values by hand, observing UART output, and iterating until all four classes were separable was a challenging but rewarding process that gave me deep insight into the fundamentals of neural inference.

Preetham V (1RV23ET030)

My primary focus was on the firmware architecture — partitioning tasks between the ARM core and the UDB, managing the I2C polling sequence, and implementing the UART logging interface. Seeing how a well-structured embedded firmware design enables two parallel inference paths to run on the same hardware without conflicts was a formative experience in real-time systems design.

Tejas K S (1RV23EE054)

I was responsible for the cycle-accurate benchmarking methodology using the on-chip 24-bit timer. Setting up a fair, independent timer that excluded peripheral overhead and measuring the same workload twice — once in hardware and once in software — and consistently observing the $2.27\times$ speedup across dozens of live samples gave me confidence in experimental embedded systems evaluation. This project has significantly shaped my interest in edge AI hardware.

10. Outcome of the Work

The following outcomes have been achieved or are in progress as a result of this Experiential Learning project:

Research Paper

A full-length IEEE-format research paper titled "Hardware-Mapped Neural Inference Engine for Real-Time Elderly Fall Detection" has been authored, documenting the system architecture, hardware MAC design, experimental methodology, and quantitative results. The paper is being prepared for submission to an IEEE journal in the area of embedded systems and biomedical engineering.

Working Prototype

A fully functional hardware prototype was developed and validated on the PSoC 5LP development board, demonstrating real-time fall detection with hardware-accelerated inference. The prototype correctly classifies four activity states under live sensor input and consistently produces a measured $2.27\times$ speedup.

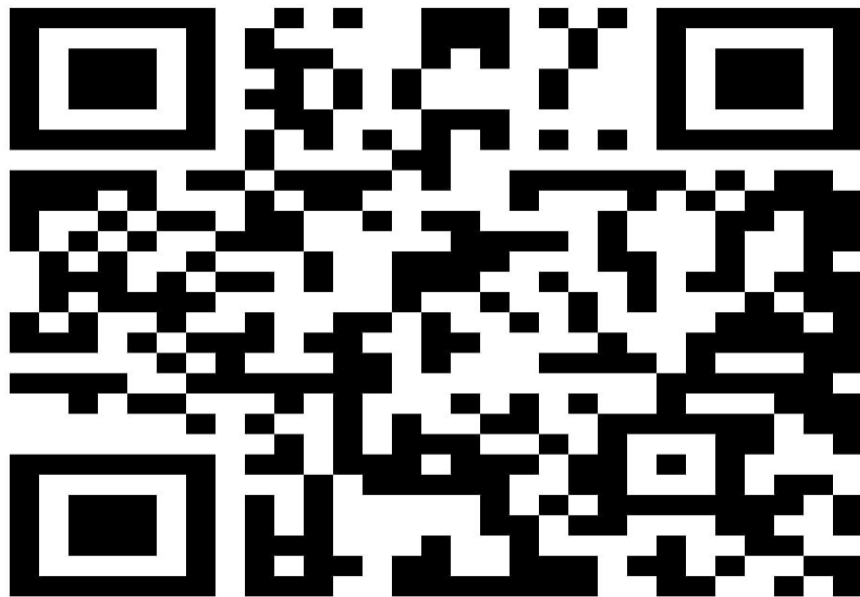
SDG-9 Alignment

This project directly contributes to United Nations Sustainable Development Goal 9 (Industry, Innovation

and Infrastructure) by developing a novel low-cost hardware infrastructure for edge AI in healthcare, demonstrating that intelligent inference can be embedded in affordable SoC-based devices without cloud infrastructure — making smart health monitoring accessible to resource-constrained settings.

Outcome Type	Description	Status
Research Paper	IEEE-format paper on hardware-mapped neural inference for fall detection	Prepared / Under Submission
Working Prototype	PSoC 5LP system with UDB MAC engine, 3 I2C sensors, UART + LED output	Completed
SDG Contribution	Low-cost edge AI infrastructure for elderly healthcare monitoring	Demonstrated
IEEE Publication	Target: IEEE Journal of Embedded Systems or IEEE Sensors Journal	In Progress

11. QR Code of Demonstration Video



12. References

- [1] World Health Organization, "Falls," WHO Fact Sheets, 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/falls>
- [2] M. Mubashir, L. Shao, and L. Seed, "A survey on fall detection: Principles and approaches," *Neurocomputing*, vol. 100, pp. 144–152, 2013.
- [3] F. Bianchi, S. J. Redmond, M. R. Narayanan, S. Cerutti, and N. H. Lovell, "Barometric pressure and triaxial accelerometry-based falls event detection," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 18, no. 6, pp. 619–627, 2010.
- [4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [5] Cypress Semiconductor Corporation, PSoC 5LP Technical Reference Manual, Document No. 001-78426, San Jose, CA, USA.
- [6] Bosch Sensortec GmbH, BMP280 Digital Pressure Sensor Datasheet, Document No. BST-BMP280-DS001, Reutlingen, Germany.
- [7] Texas Instruments, INA219 Zero-Drift, Bidirectional Current/Power Monitor with I2C Interface Datasheet, Document No. SBOS448, Dallas, TX, USA.
- [8] G. Santhanamari et al., "A Smart Wearable-Based Fall Detection and Health Monitoring System for Elderly Care Using IoT and Machine Learning," *IEEE / ICNGCS*, 2025.
- [9] M. S. Diab and E. Rodriguez-Villegas, "Embedded Machine Learning Using Microcontrollers in Wearable and Ambulatory Systems for Health and Care Applications: A Review," *IEEE Access*, 2022.
- [10] S. Kiran et al., "Unveiling Fall Origins: Leveraging Wearable Sensors to Detect Pre-Impact Fall Causes," *IEEE Sensors Journal*, 2024.
- [11] Y. Sun, X. Liu, and W. Zhang, "A Survey on Fall Detection Systems: Principles, Methods and Applications," *IEEE Access*, Vol. 8, pp. 128479–128497, 2020.
- [12] InvenSense, "MPU6050 Product Specification," Rev. 4.2, 2020.